University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 2

Date: Feb. 20th, 2026

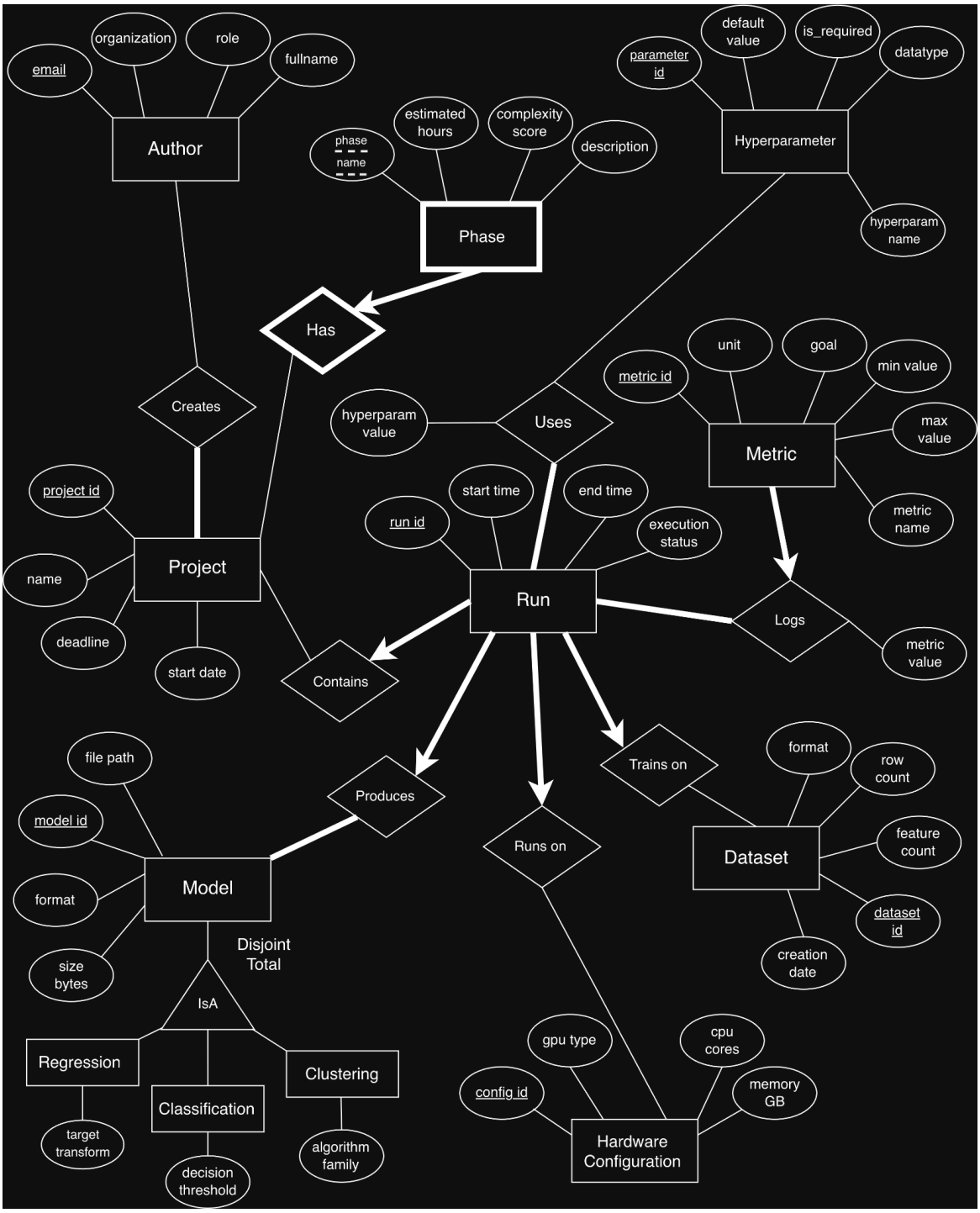
Group Number: 26

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Mark Skrypnyk	86867629	p5k9k	skrypnykmark@gmail.com
Chaitanya Thakral	22283816	r1l4x	cthakral6@gmail.com
Adi Yadav	80981251	b4l2k	aditya_1007@icloud.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia.

Our project is a database for machine learning experiment tracking and model management. It records projects and phases, and tracks each experimental run with its dataset, hyperparameters, hardware configuration, produced model, and performance metrics. The goal is to make runs easy to compare and ensure results are traceable and reproducible over time. The application will be written in TypeScript, React for the front end and Node.js/Express for the backend. We will use the department provided Oracle database. Below is our ER diagram that we are basing our application on:



Relational Schema
Note: underscores were used in place of spaces in the ER diagram to be consistent with SQL naming.

Author(email: VARCHAR(100), organization: VARCHAR(100), role: VARCHAR(100), fullname: VARCHAR(100))
- PK: email

Project(project_id: INTEGER, name: VARCHAR(100), deadline: DATE, start_date: DATE)
- PK: project_id

Phase(phase_name: VARCHAR(100), project_id: INTEGER, estimated_hours: FLOAT, complexity_score: INTEGER, description: VARCHAR(500))
- PK: phase_name, project_id
- FK: project_id references Project(project_id)
- Constraints: project_id NOT NULL

Hyperparameter(parameter_id: INTEGER, hyperparam_name: VARCHAR(100), default_value: VARCHAR(100), is_required: CHAR(1), datatype: VARCHAR(100))
- PK: parameter_id
- CK: hyperparam_name
- Constraints: hyperparam_name NOT NULL UNIQUE

Dataset(dataset_id: INTEGER, format: VARCHAR(100), row_count: INTEGER, feature_count: INTEGER, creation_date: DATE)
- PK: dataset_id

HardwareConfiguration(config_id: INTEGER, gpu_type: VARCHAR(100), cpu_cores: INTEGER, memory_GB: FLOAT)
- PK: config_id

Model(model_id: INTEGER, file_path: VARCHAR(255), format: VARCHAR(100), size_bytes: INTEGER)
- PK: model_id
- CK: file_path
- Constraints: file_path NOT NULL UNIQUE

Regression(model_id: INTEGER, target_transform: VARCHAR(100))
- PK: model_id
- FK: model_id references Model(model_id)

Classification(model_id: INTEGER, decision_threshold: FLOAT)
- PK: model_id
- FK: model_id references Model(model_id)

Clustering(model_id: INTEGER, algorithm_family: VARCHAR(100))

- PK: model_id
- FK: model_id references Model(model_id)

Metric(metric_id: INTEGER, metric_name: VARCHAR(100), unit: VARCHAR(100), goal: VARCHAR(100), min_value: FLOAT, max_value: FLOAT, run_id: INTEGER, metric_value: FLOAT)

- PK: metric_id
- FK: run_id references Run(run_id)
- Constraints: run_id NOT NULL

Run(run_id: INTEGER, start_time: TIMESTAMP, end_time: TIMESTAMP, execution_status: VARCHAR(20), project_id: INTEGER, model_id: INTEGER, dataset_id: INTEGER, config_id: INTEGER)

- PK: run_id
- FK: project_id references Project(project_id)
- FK: model_id references Model(model_id)
- FK: config_id references HardwareConfiguration(config_id)
- FK: dataset_id references Dataset(dataset_id)
- Constraints: project_id NOT NULL, model_id NOT NULL, config_id NOT NULL, dataset_id NOT NULL

Creates(email: VARCHAR(100), project_id: INTEGER)

- PK: email, project_id
- FK: email references Author(email)
- FK: project_id references Project(project_id)

Uses(run_id: INTEGER, parameter_id: INTEGER, hyperparam_value: VARCHAR(100))

- PK: run_id, parameter_id
- FK: run_id references Run(run_id)
- FK: parameter_id references Hyperparameter(parameter_id)

Functional Dependencies

Author

- email → organization, role, fullname (PK)

Project

- project_id → name, deadline, start_date (PK)

Phase

- phase_name, project_id → estimated_hours, complexity_score, description (PK)
- phase_name → complexity_score (non-PK/CK) (standard ML phases have known complexity levels)

Hyperparameter

- parameter_id → hyperparam_name, default_value, is_required, datatype (PK)
- hyperparam_name → parameter_id, default_value, is_required, datatype (CK)

Dataset
- dataset_id → format, row_count, feature_count, creation_date (PK)

HardwareConfiguration
- config_id → gpu_type, cpu_cores, memory_GB (PK)

Model
- model_id → file_path, format, size_bytes (PK)
- file_path → model_id, format, size_bytes (CK)

Regression
- model_id → target_transform (PK)

Classification
- model_id → decision_threshold (PK)

Clustering
- model_id → algorithm_family (PK)

Metric
- metric_id → metric_name, unit, goal, min_value, max_value, run_id, metric_value (PK)
- metric_name → unit (non-PK/CK) (e.g. accuracy always uses a decimal)
- metric_name → goal (non-PK/CK) (e.g. minimize loss)
- metric_name → min_value (non-PK/CK) (accuracy always has min value 0)
- metric_name → max_value (non-PK/CK) (accuracy always has max value 1)

Run
- run_id → start_time, end_time, execution_status, project_id, model_id, dataset_id, config_id (PK)

Creates
- no non-trivial FDs

Uses
- run_id, parameter_id → hyperparam_value (PK)

Normalization

Author
- Author(email: VARCHAR(100), organization: VARCHAR(100), role: VARCHAR(100), fullname: VARCHAR(100))
    - PK: email

- FD: email → organization, role, fullname
- email+ = {email, organization, role, fullname} = all attributes, email is a superkey
- Already in BCNF

Project

- Project(project_id: INTEGER, name: VARCHAR(100), deadline: DATE, start_date: DATE)
    - PK: project_id
- FD: project_id → name, deadline, start_date
- project_id+ = {project_id, name, deadline, start_date} = all attributes, project_id is a superkey
- Already in BCNF

Phase

- Phase(phase_name: VARCHAR(100), project_id: INTEGER, estimated_hours: FLOAT, complexity_score: INTEGER, description: VARCHAR(500))
    - PK: phase_name, project_id
    - FK: project_id references Project(project_id)
- FDs are:
    - phase_name, project_id → estimated_hours, complexity_score, description
    - phase_name → complexity_score
- phase_name+ = {phase_name, complexity_score} ≠ all attributes, phase_name not a superkey
- BCNF violated because phase_name is not a superkey, so we decompose:
- R1(phase_name, project_id, estimated_hours, description), R2(phase_name, complexity_score)
- Check R1: (phase_name, project_id)+ = {phase_name, project_id, estimated_hours, description} = all attributes of R1, (phase_name, project_id) is a superkey
- Check R2: phase_name+ = {phase_name, complexity_score} = all attributes of R2, phase_name is a superkey
- Now we construct our final tables:
- PhaseComplexity(phase_name: VARCHAR(100), complexity_score: INTEGER)
    - PK: phase_name
- Phase(phase_name: VARCHAR(100), project_id: INTEGER, estimated_hours: FLOAT, description: VARCHAR(500))
    - PK: phase_name, project_id
    - FK: phase_name references PhaseComplexity(phase_name)
    - FK: project_id references Project(project_id)

Hyperparameter

- Hyperparameter(parameter_id: INTEGER, hyperparam_name: VARCHAR(100), default_value: VARCHAR(100), is_required: CHAR(1), datatype: VARCHAR(100))
    - PK: parameter_id
    - CK: hyperparam_name
- FDs are:
    - parameter_id → hyperparam_name, default_value, is_required, datatype
    - hyperparam_name → parameter_id, default_value, is_required, datatype

- parameter_id+ = {parameter_id, hyperparam_name, default_value, is_required, datatype} = all attributes, parameter_id is a superkey
- hyperparam_name+ = {parameter_id, hyperparam_name, default_value, is_required, datatype} = all attributes, hyperparam_name is a superkey
- Already in BCNF

Dataset

- Dataset(dataset_id: INTEGER, format: VARCHAR(100), row_count: INTEGER, feature_count: INTEGER, creation_date: DATE)
    - PK: dataset_id
- FD: dataset_id → format, row_count, feature_count, creation_date
- dataset_id+ = {dataset_id, format, row_count, feature_count, creation_date} = all attributes, dataset_id is a superkey
- Already in BCNF

HardwareConfiguration

- HardwareConfiguration(config_id: INTEGER, gpu_type: VARCHAR(100), cpu_cores: INTEGER, memory_GB: FLOAT)
    - PK: config_id
- FD: config_id → gpu_type, cpu_cores, memory_GB
- config_id+ = {config_id, gpu_type, cpu_cores, memory_GB} = all attributes, config_id is a superkey
- Already in BCNF

Model

- Model(model_id: INTEGER, file_path: VARCHAR(255), format: VARCHAR(100), size_bytes: INTEGER)
    - PK: model_id
    - CK: file_path
- FDs are:
    - model_id → file_path, format, size_bytes
    - file_path → model_id, format, size_bytes
- model_id+ = {model_id, file_path, format, size_bytes} = all attributes, model_id is a superkey
- file_path+ = {model_id, file_path, format, size_bytes} = all attributes, file_path is a superkey
- Already in BCNF

Regression

- Regression(model_id: INTEGER, target_transform: VARCHAR(100))
    - PK: model_id
    - FK: model_id references Model(model_id)
- FD: model_id → target_transform
- model_id+ = {model_id, target_transform} = all attributes, model_id is a superkey
- Already in BCNF

Classification

- Classification(model_id: INTEGER, decision_threshold: FLOAT)
    - PK: model_id
    - FK: model_id references Model(model_id)
- FD: model_id → decision_threshold
- model_id+ = {model_id, decision_threshold} = all attributes, model_id is a superkey
- Already in BCNF

Clustering

- Clustering(model_id: INTEGER, algorithm_family: VARCHAR(100))
    - PK: model_id
    - FK: model_id references Model(model_id)
- FD: model_id → algorithm_family
- model_id+ = {model_id, algorithm_family} = all attributes, model_id is a superkey
- Already in BCNF

Metric

- Metric(metric_id: INTEGER, metric_name: VARCHAR(100), unit: VARCHAR(100), goal: VARCHAR(100), min_value: FLOAT, max_value: FLOAT, run_id: INTEGER, metric_value: FLOAT)
    - PK: metric_id
    - FK: run_id references Run(run_id)
- FDs are:
    - metric_id → metric_name, unit, goal, min_value, max_value, run_id, metric_value
    - metric_name → unit
    - metric_name → goal
    - metric_name → min_value
    - metric_name → max_value
- Using union rule: metric_name → unit, goal, min_value, max_value
- metric_name+ = {metric_name, unit, goal, min_value, max_value} ≠ all attributes, metric_name is not a superkey
- BCNF violated because metric_name is not a superkey, so we decompose:
- R1(metric_name, unit, goal, min_value, max_value), R2(metric_id, metric_name, run_id, metric_value)
- Check R1: metric_name+ = {metric_name, unit, goal, min_value, max_value} = all attributes of R1, metric_name is a superkey
- Check R2: metric_id+ = {metric_id, metric_name, run_id, metric_value} = all attributes of R2, metric_id is a superkey
- Now we construct our final tables:
- MetricType(metric_name: VARCHAR(100), unit: VARCHAR(100), goal: VARCHAR(100), min_value: FLOAT, max_value: FLOAT)
    - PK: metric_name
- Metric(metric_id: INTEGER, metric_name: VARCHAR(100), run_id: INTEGER, metric_value: FLOAT)

    - PK: metric_id
    - FK: metric_name references MetricType(metric_name)
    - FK: run_id references Run(run_id)

Run

- Run(run_id: INTEGER, start_time: TIMESTAMP, end_time: TIMESTAMP, execution_status: VARCHAR(20), project_id: INTEGER, model_id: INTEGER, dataset_id: INTEGER, config_id: INTEGER)
    - PK: run_id
    - FK: project_id references Project(project_id)
    - FK: model_id references Model(model_id)
    - FK: config_id references HardwareConfiguration(config_id)
    - FK: dataset_id references Dataset(dataset_id)
- FD: run_id → start_time, end_time, execution_status, project_id, model_id, dataset_id, config_id
- run_id+ = {run_id, start_time, end_time, execution_status, project_id, model_id, dataset_id, config_id} = all attributes, run_id is a superkey
- Already in BCNF

Creates

- Creates(email: VARCHAR(100), project_id: INTEGER)
    - PK: email, project_id
    - FK: email references Author(email)
    - FK: project_id references Project(project_id)
- FD: no non-trivial FDs to violate BCNF
- already in BCNF

Uses

- Uses(run_id: INTEGER, parameter_id: INTEGER, hyperparam_value: VARCHAR(100))
    - PK: run_id, parameter_id
    - FK: run_id references Run(run_id)
    - FK: parameter_id references Hyperparameter(parameter_id)

- FD: (run_id, parameter_id) → hyperparam_value
- (run_id, parameter_id)+ = {run_id, parameter_id, hyperparam_value} = all attributes, (run_id, parameter_id) is a superkey
- already in BCNF

Only Phase was decomposed into PhaseComplexity, Phase and Metric was decomposed into MetricType, Metric, all others were already in BCNF and remain unchanged.

SQL DDL statements below…

SQL DDL Statements

Note: Oracle does not support ON UPDATE CASCADE as mentioned in the project description and is not included.

```
CREATE TABLE Author (
        email           VARCHAR(100),
        organization    VARCHAR(100),
        role            VARCHAR(100),
        fullname        VARCHAR(100),
        PRIMARY KEY (email)
);

CREATE TABLE Project (
        project_id              INTEGER,
        name                    VARCHAR(100),
        deadline                DATE,
        start_date              DATE,
        PRIMARY KEY (project_id)
);

CREATE TABLE PhaseComplexity (
        phase_name              VARCHAR(100),
        complexity_score        INTEGER,
        PRIMARY KEY (phase_name)
);

CREATE TABLE Phase (
        phase_name              VARCHAR(100),
        project_id              INTEGER,
        estimated_hours         FLOAT,
        description             VARCHAR(500),
        PRIMARY KEY (phase_name, project_id),
        FOREIGN KEY  (phase_name)
                REFERENCES PhaseComplexity(phase_name)
                ON DELETE CASCADE,
        FOREIGN KEY  (project_id)
                REFERENCES Project(project_id)
                ON DELETE CASCADE
);

CREATE TABLE Hyperparameter (
        parameter_id            INTEGER,
```

```
        hyperparam_name     VARCHAR(100),
        default_value       VARCHAR(100),
        is_required         CHAR(1),
        datatype            VARCHAR(100),
        PRIMARY KEY (parameter_id),
        UNIQUE (hyperparam_name)
);

CREATE TABLE Dataset (
        dataset_id      INTEGER,
        format          VARCHAR(100),
        row_count       INTEGER,
        feature_count   INTEGER,
        creation_date   DATE,
        PRIMARY KEY (dataset_id)
);

CREATE TABLE HardwareConfiguration (
        config_id       INTEGER,
        gpu_type        VARCHAR(100),
        cpu_cores       INTEGER,
        memory_GB       FLOAT,
        PRIMARY KEY (config_id)
);

CREATE TABLE Model (
        model_id        INTEGER,
        file_path       VARCHAR(255),
        format          VARCHAR(100),
        size_bytes      INTEGER,
        PRIMARY KEY (model_id),
        UNIQUE (file_path)
);

CREATE TABLE Regression (
        model_id            INTEGER,
        target_transform    VARCHAR(100),
        PRIMARY KEY (model_id),
        FOREIGN KEY  (model_id)
                REFERENCES Model(model_id)
                ON DELETE CASCADE
);

CREATE TABLE Classification (
```

```
        model_id                INTEGER,
        decision_threshold      FLOAT,
        PRIMARY KEY (model_id),
        FOREIGN KEY  (model_id)
                REFERENCES Model(model_id)
                ON DELETE CASCADE
);

CREATE TABLE Clustering (
        model_id                INTEGER,
        algorithm_family        VARCHAR(100),
        PRIMARY KEY (model_id),
        FOREIGN KEY  (model_id)
                REFERENCES Model(model_id)
                ON DELETE CASCADE
);

CREATE TABLE MetricType (
        metric_name     VARCHAR(100),
        unit            VARCHAR(100),
        goal            VARCHAR(100),
        min_value       FLOAT,
        max_value       FLOAT,
        PRIMARY KEY (metric_name)
);

CREATE TABLE Run (
        run_id                  INTEGER,
        start_time              TIMESTAMP NULL,
        end_time                TIMESTAMP NULL,
        execution_status        VARCHAR(20),
        project_id              INTEGER NOT NULL,
        model_id                INTEGER NOT NULL,
        dataset_id              INTEGER NOT NULL,
        config_id               INTEGER NOT NULL,
        PRIMARY KEY (run_id),
        FOREIGN KEY  (project_id)
                REFERENCES Project(project_id)
                ON DELETE CASCADE,
        FOREIGN KEY  (model_id)
                REFERENCES Model(model_id)
                ON DELETE CASCADE,
        FOREIGN KEY  (dataset_id)
                REFERENCES Dataset(dataset_id)
```

```
        ON DELETE CASCADE,
    FOREIGN KEY  (config_id)
        REFERENCES HardwareConfiguration(config_id)
        ON DELETE CASCADE
);

CREATE TABLE Metric (
    metric_id       INTEGER,
    metric_name     VARCHAR(100)        NOT NULL,
    run_id          INTEGER             NOT NULL,
    metric_value    FLOAT,
    PRIMARY KEY (metric_id),
    FOREIGN KEY  (metric_name)
        REFERENCES MetricType(metric_name)
        ON DELETE CASCADE,
    FOREIGN KEY  (run_id)
        REFERENCES Run(run_id)
        ON DELETE CASCADE
);

CREATE TABLE Creates (
    email           VARCHAR(100),
    project_id      INTEGER,
    PRIMARY KEY (email, project_id),
    FOREIGN KEY  (email)
        REFERENCES Author(email)
        ON DELETE CASCADE,
    FOREIGN KEY  (project_id)
        REFERENCES Project(project_id)
        ON DELETE CASCADE
);

CREATE TABLE Uses (
    run_id              INTEGER,
    parameter_id        INTEGER,
    hyperparam_value    VARCHAR(100),
    PRIMARY KEY (run_id, parameter_id),
    FOREIGN KEY  (run_id)
        REFERENCES Run(run_id)
        ON DELETE CASCADE,
    FOREIGN KEY  (parameter_id)
        REFERENCES Hyperparameter(parameter_id)
        ON DELETE CASCADE
);
```

SQL DDL Foreign Key Handling

- Phase → Project: If a project is deleted, its phases are no longer relevant and should be removed with it.
- Phase → PhaseComplexity: A phase's name is defined by its complexity category, if the category no longer exists, the phase it defines no longer has any meaning and should be removed with it.
- Regression, Classification, Clustering → Model: Since these are subtypes of a model, if the model is deleted, the subtypes are no longer relevant.
- Run → Project, Model, Dataset, HardwareConfiguration: a run cannot exist separate of all four of these, the run record becomes meaningless if any is deleted.
- Metric → MetricType, Run: A metric value only makes sense in the context of its type and its run so deleting any of these should clean up the associated metrics.
- Creates → Author, Project: The relationship record has no meaning if either the author or project don't exist.
- Uses → Run, Hyperparameter: A hyperparameter usage record is meaningless without its associated run or hyperparameter definition.

AI TOOLS WERE NOT USED IN THIS ASSIGNMENT.